실습 가이드 매뉴얼

Claude Code × Blender 3D 모션그래픽

전기차 조립 공정 영상을 코드로 만든다

GLB 모델에서 1080p MP4까지. 스케이트보드 30초 영상에 이어 차체를 더한 전체 공정 53초 영상까지 따라간다

수강생은 프롬프트를 붙여넣고 결과를 확인한다. 설치, 코드 작성, 렌더링, 오류 해결은 모두 Claude Code가 한다.
산출물: scene.py와 ev2_scene.py(장면), 자막 합성과 효과음 스크립트, 최종 MP4 두 편

3D 모션그래픽은 코드로 만든 장면을 렌더링한 결과다

이번 강의에서 가져갈 세 가지 메시지

1

키프레임을 손으로 찍지 않는다

부품마다 출발점과 도착점만 정하면 Blender가 그 사이를 보간한다. 이 규칙을 파이썬 코드로 쓴다.

2

사람은 기획하고 판정하며, Claude Code는 만든다

무엇을 보여줄지와 미리보기 판정은 사람이 하고, 설치부터 인코딩까지는 Claude Code에 맡긴다.

3

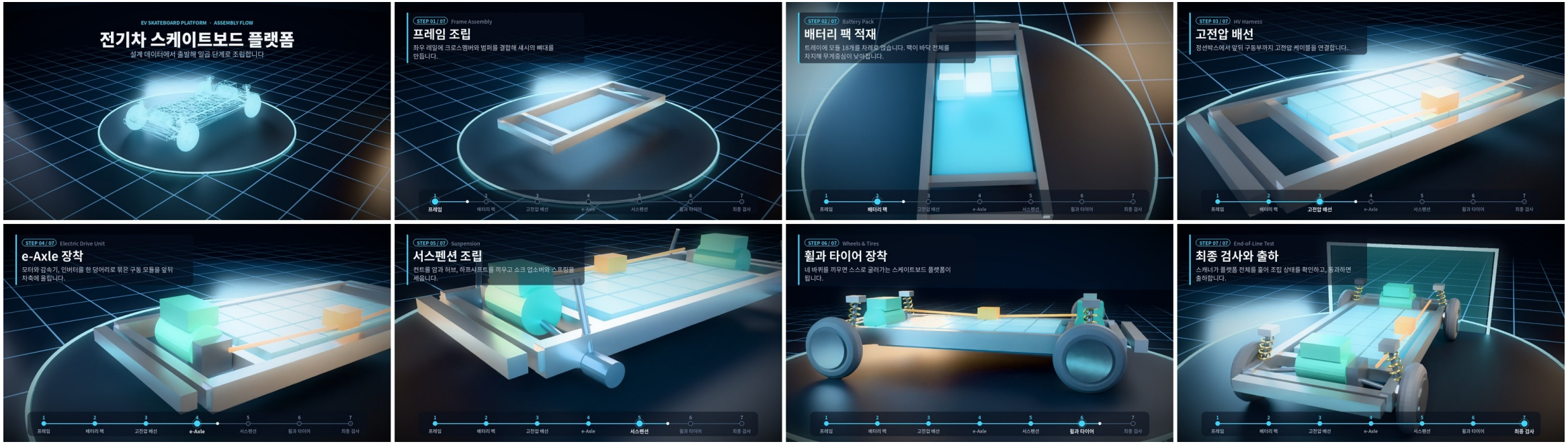
미리보기로 구도를 확정한 뒤에 본 렌더링을 건다

본 렌더링은 CPU에서 몇 시간이 걸린다. 저해상도 정지컷 12장으로 먼저 고칠 점을 찾는다.

결과물: 스케이트보드 7단계 30초 영상과, 차체를 더한 9단계 53초 전체 공정 영상

완성 영상은 일곱 단계 조립 공정을 30초에 담았다

왼쪽 위에서 오른쪽 아래 순서로 홀로그램, 프레임, 배터리, 배선, e-Axle, 서스펜션, 휠, 검사



용어 여섯 개를 알면 전체 흐름을 읽을 수 있다

처음 나오는 용어의 뜻과 이번 실습에서의 쓰임

용어	뜻	이번 실습에서는
Blender와 bpy	무료 3D 프로그램, bpy는 그 파이썬 모듈	화면 없이 스크립트로 Blender 실행
GLB(glTF)	3D 모델을 한 파일에 담는 표준 형식	부품 408개짜리 EV 섀시 모델
키프레임과 보간	특정 프레임의 값을 기록하고 그 사이를 채움	출발 위치와 도착 위치 두 개만 기록
렌더 엔진	장면을 이미지로 계산하는 방식	Cycles(광선 추적, CPU 가능) 사용
샘플과 디노이즈	픽셀당 광선 수, 남은 노이즈를 지우는 필터	5~8샘플 + OpenImageDenoise
FFmpeg	영상과 소리를 변환하고 합치는 도구	PNG 720장과 WAV를 MP4로 인코딩

코드로 만든 영상은 고치고 다시 만들기가 쉽다

같은 30초 영상을 수작업과 코드로 만들 때의 차이

부품의 위치를 바꾸거나 단계 순서를 바꿔도 스크립트 한 줄을 고치고 다시 렌더링하면 된다

수작업 애니메이션

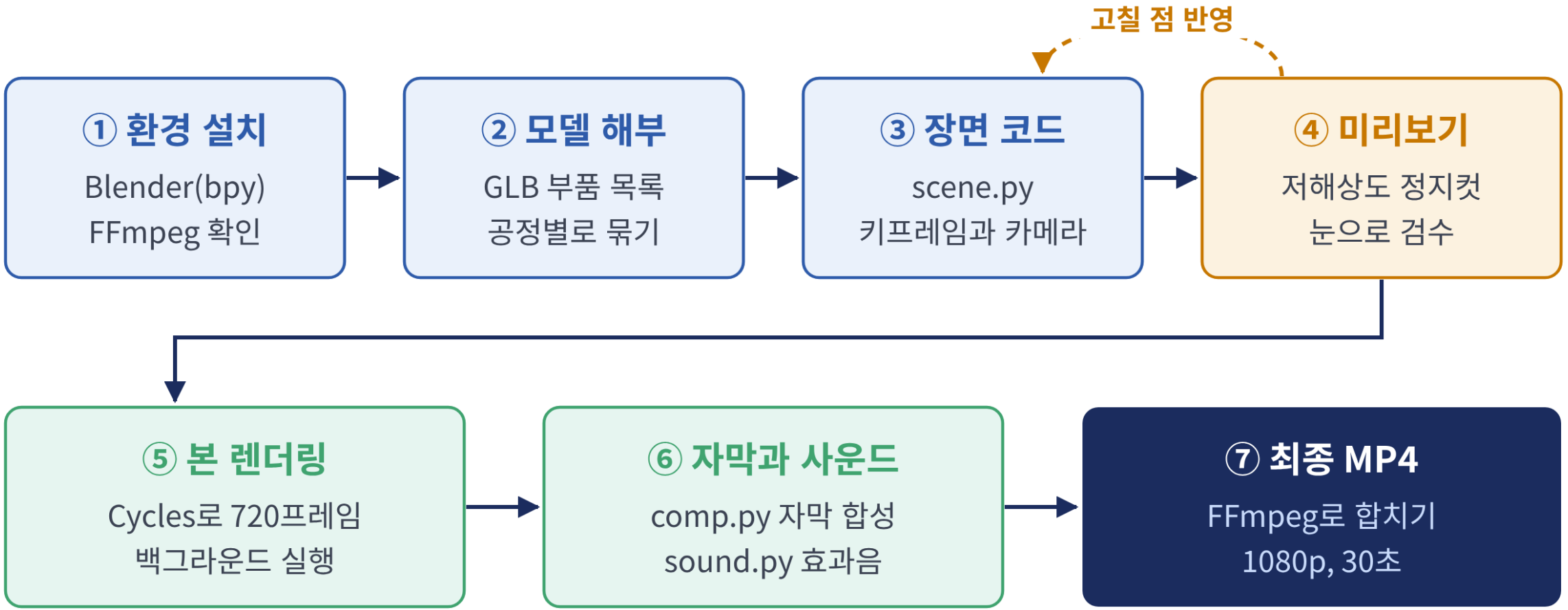
- 408개 부품마다 키프레임을 손으로 찍는다
- 순서를 바꾸면 뒤 키프레임을 모두 옮긴다
- 다른 모델에 재사용하기 어렵다

코드 애니메이션

- 이름 패턴으로 부품을 묶어 한 번에 움직인다
- 시작 프레임 숫자만 바꾸면 순서가 바뀐다
- 다른 GLB 모델에도 같은 함수를 쓴다

제작은 일곱 단계로 진행하고 미리보기에서 한 번 되돌아간다

파란 단계는 준비, 앰버는 검수, 초록은 시간이 오래 걸리는 생산 단계다



01

PART 1 · 준비

환경 설치와 모델 해부

Blender를 설치하고, GLB 모델이 어떤 부품으로 이뤄졌는지 파악한다

필요한 도구는 네 가지이며 모두 무료다

GPU가 있으면 렌더링이 수십 배 빨라지지만, 없어도 끝까지 만들 수 있다

도구	역할	설치 방법
Claude Code	설치, 코드 작성, 실행, 오류 해결 전부	수강 전 설치 완료
Blender 4.x 이상	3D 장면 구성과 렌더링	앱 설치, 또는 pip install bpy
Python	Blender 스크립트와 자막, 소리 합성	Claude Code가 확인
FFmpeg	프레임과 소리를 MP4로 합치기	Claude Code가 설치

준비할 소재는 3D 모델 한 개

- 부품이 이름별로 나뉜 GLB 파일. 이번 실습은 EV_Skateboard_Education.glb

STEP 1

환경 점검과 렌더 엔진 시험을 한 번에 맡긴다

엔진마다 렌더가 되는지와 걸리는 시간을 먼저 재야 영상 길이를 정할 수 있다

[입력] 프롬프트 · Claude Code

```
이 폴더에서 Blender로 3D 모션그래픽을
만들려고 한다.

1) blender, ffmpeg, python 버전을 확인해줘.
2) blender가 없으면 pip로 bpy를 설치하고
   import bpy 후 버전을 출력해 확인해줘.
3) Eevee, Workbench, Cycles로 각각 한 장씩
   시험 렌더해서 성공 여부와 걸린 시간을
   표로 알려줘.
4) 오류가 나면 원인을 찾아 직접 고쳐줘.
```

[출력] 결과 · 환경 점검

```
Blender 5.2.3 LTS (bpy 설치)
FFmpeg 있음, Python 3.13

Eevee       실패
  GPU가 없어 EGL 오류
Workbench   실패
  같은 이유
Cycles      성공
  960×540, 16샘플, 13초
```

부품 이름이 곧 조립 공정의 단서다

GLB를 불러와 부품 이름, 재질, 위치 범위를 뽑으면 공정 단계로 묶을 수 있다

[입력] 프롬프트 · Claude Code

```
model.glb를 Blender로 불러와 부품 목록을
뽑아줘.

- 부품 이름, 꼭짓점 수, 재질, 위치 범위
- 이름 패턴으로 공정 단계에 묶기:
  프레임, 배터리, 배선, 구동, 서스펜션, 휠
- 차의 앞쪽 방향과 바닥 높이도 알려줘

스크립트 파일 이름은 파이썬 표준 모듈과
겹치지 않게 지어줘.
```

[출력] 결과 · 부품 분류

```
메시 408개
(스프링 코일 조각 336개 포함)

프레임: 레일 2, 크로스멤버 4,
        범퍼 2
배터리: 트레이 1, 모듈 18(3×6)
배선:   정션박스 1, 케이블 2
구동:   모터, 감속기, 인버터 앞뒤
서스펜션: 암 8, 허브 4, 샤프트 4
휠:     타이어 4, 림 8
앞쪽 = -Y, 타이어 바닥 z = 0
```

부품 묶음마다 등장 방식과 시간을 배정한다

24fps 기준, 프레임 번호를 24로 나누면 초가 된다(예: 240프레임 = 10초)

단계	부품	등장 방식	프레임
1 프레임	레일, 크로스멤버, 범퍼	좌우에서 밀려 들어오고 위에서 떨어진다	62~134
2 배터리 팩	트레이, 모듈 18개	트레이가 바닥에서 솟고 모듈이 차례로 낙하	144~248
3 고전압 배선	정션박스, 케이블 2	박스가 떨어지고 케이블이 길이 방향으로 자란다	262~308
4 e-Axle	모터, 감속기, 인버터	아래에서 솟고 옆과 위에서 결합한다	322~392
5 서스펜션	암, 허브, 샤프트, 댐퍼, 스프링	바깥에서 안으로, 위에서 아래로	402~456
6 휠과 타이어	타이어 4, 림 8	두 바퀴 회전하며 옆에서 끼워진다	482~526
7 최종 검사	스캐너 게이트, 턴테이블	앞뒤로 훑은 뒤 0.6바퀴 회전	562~720

02

PART 2 · 장면

애니메이션 원리와 장면 스크립트

fly_in 함수 하나로 408개 부품을 움직이고, 연출 효과를 더한다

부품의 움직임은 키프레임 두 개와 감속 곡선으로 정해진다

시작 프레임에 오프셋 위치, 끝 프레임에 최종 위치를 기록하면 그 사이는 곡선이 채운다

$$\underbrace{p(t)}_{\text{현재 위치}} = \underbrace{p_{\text{최종}}}_{\text{조립된 자리}} + (1 - e(\tau)) \underbrace{\Delta}_{\text{출발 오프셋}}$$

$$e(\tau) = 1 - 2^{-10\tau}, \qquad \tau = \frac{t - t_{\text{시작}}}{T_{\text{길이}}}$$

EXPO 보간

- 처음에 빠르게 날아와 끝에서 부드럽게 멈춘다

BACK 보간

- 목표를 살짝 지나쳤다 돌아와 결합감을 준다

장면 스크립트는 기획 내용을 메타프롬프트로 설명해 받는다

수강생은 Blender API를 몰라도 된다. 원하는 연출을 문장으로 적는다

[입력] 프롬프트 · Claude Code (메타프롬프트)

```
model.glb로 EV 스케이트보드 조립 공정 영상을
만드는 Blender 스크립트 scene.py를 작성해줘.
- 24fps, 30초(720프레임), 1280×720, Cycles
- 부품은 최종 자리에서 떨어진 곳에서 출발해
  EXPO 또는 BACK 보간으로 날아와 결합
- 순서: 프레임, 배터리, 배선, e-Axle,
  서스펜션, 휠, 최종 검사(스캐너, 턴테이블)
- 모듈은 착지할 때 빛나고, 케이블은 자라나게
- 시작은 홀로그램 와이어프레임, 바닥은 발광 그리드
- 카메라는 단계마다 해당 부위를 비추게
- 효과음 타이밍을 events.json으로 저장
- preview 모드: 12프레임을 640×360으로 렌더
```

[출력] Claude가 생성 · scene.py

```
recenter()
  부품 원점을 자기 중심으로
join()
  스프링 조각 84개를 하나로
fly_in()
  오프셋에서 최종 위치로
flash()
  착지 순간 발광 키프레임
CAM = [...]
  카메라 키 17개
events.json
  효과음 큐 목록
```

다섯 가지 연출이 공정의 흐름을 눈에 보이게 만든다

모두 scene.py 안의 짧은 코드 블록이며, 메타프롬프트 한 줄씩으로 요청했다

홀로그램

0~90프레임

설계 데이터를 와이어프레임으로 먼저 보여주고 서서히 사라진다

착지 플래시

배터리 모듈

모듈마다 재질을 복제해 착지 순간 발광 강도를 올렸다 내린다

케이블 성장

고전압 배선

원점을 정션박스 쪽 끝으로 옮기고 길이 방향 크기를 0에서 1로 키운다

스캐너 게이트

최종 검사

반투명 발광 판이 앞에서 뒤로 지나가며 차체를 훑는다

링 색 변화

검사 통과

바닥 링이 청록에서 초록으로 바뀌고 턴테이블이 돈다

03

PART 3 · 렌더링

미리보기와 본 렌더링

엔진을 고르고, 시간을 계산하고, 정지컷으로 검수한 뒤 본 렌더링을 건다

GPU가 없으면 Cycles CPU가 현실적인 선택이다

1280×720 한 프레임 기준 실측값이다. Eevee는 원래 GPU 엔진이다

실패	61초	19초	13초
Eevee GPU가 없어 EGL 오류	Eevee + 가상 디스플레이 소프트웨어 GL	Cycles 8샘플 홀로그램 구간 포함	Cycles 5샘플 반사 1회로 줄인 설정

판단 기준

- GPU가 있는 PC라면 Eevee로 프레임당 1초 안팎이라 몇 분이면 끝난다. 클라우드 CPU에서는 샘플 수가 시간을 결정한다.

렌더 시간은 프레임 수와 샘플 수가 결정한다

1샘플과 5샘플을 각각 재서 고정 비용 약 4초, 샘플당 약 1.8초를 구했다

$$T_{\text{전체}} = N_{\text{프레임}} \times \left(t_{\text{고정}} + S_{\text{샘플 수}} \times t_{\text{샘플당}} \right)$$

$$630 \times \left(4 + 5 \times 1.8 \right) = 8{,}190 \text{ 초} \approx 2.3 \text{ 시간}$$

고정 비용 t고정

- 장면 갱신과 디노이즈. 샘플을 줄여도 남는다

샘플 비용 S × t샘플당

- 샘플을 8에서 5로 줄이면 프레임당 약 5초 절약

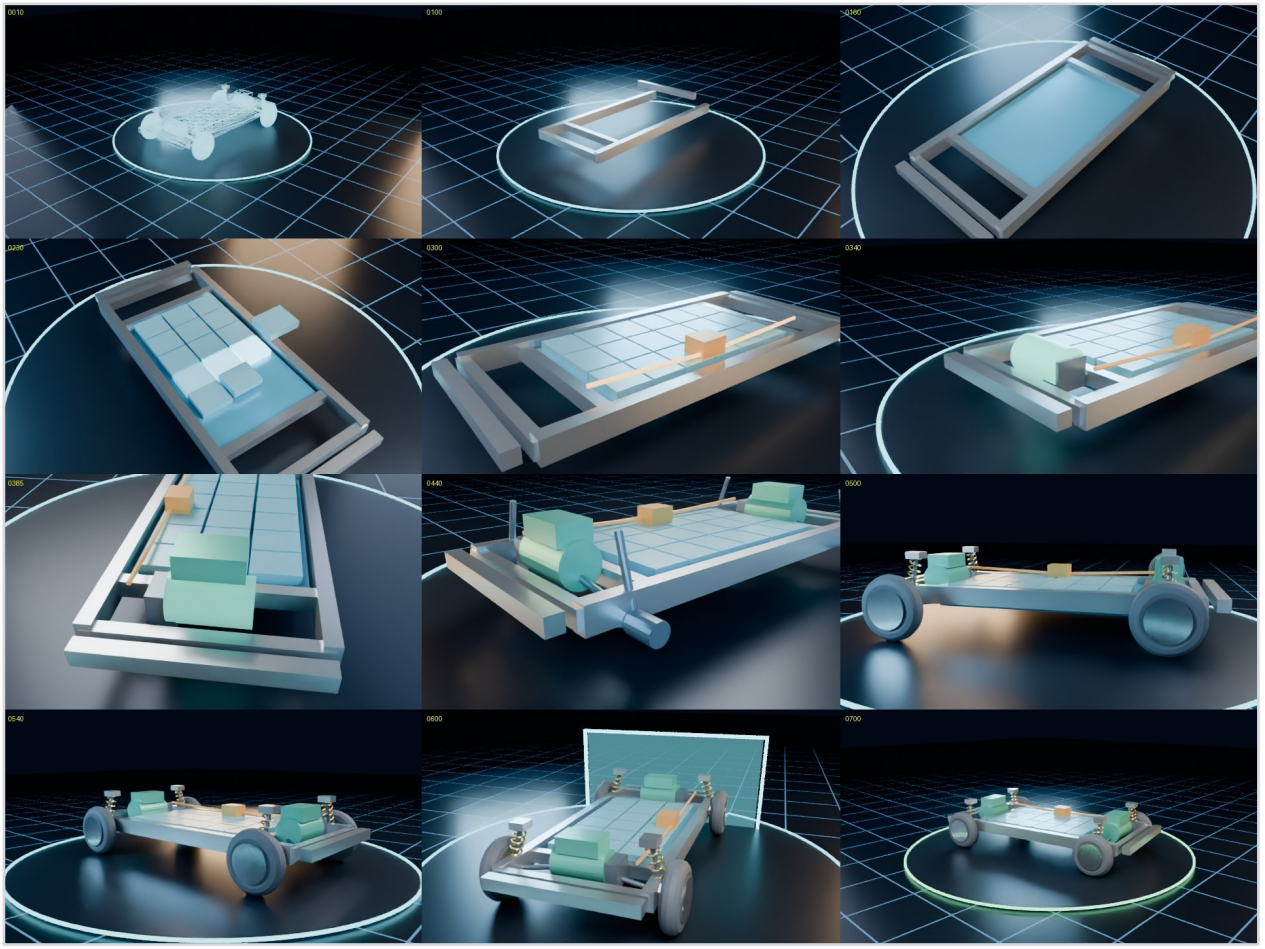
STEP 4

미리보기 12장에서 세 가지 문제를 찾았다

640×360, 6샘플로 주요 프레임만 렌더하면 1분 안에 전체 구도를 본다

발견한 문제

- 색이 바랜다
- 배터리 단계에서 차 뒤쪽이 잘린다
- 스캐너가 벽처럼 보인다



STEP 4

고칠 점은 관찰한 그대로 적어 넘기면 된다

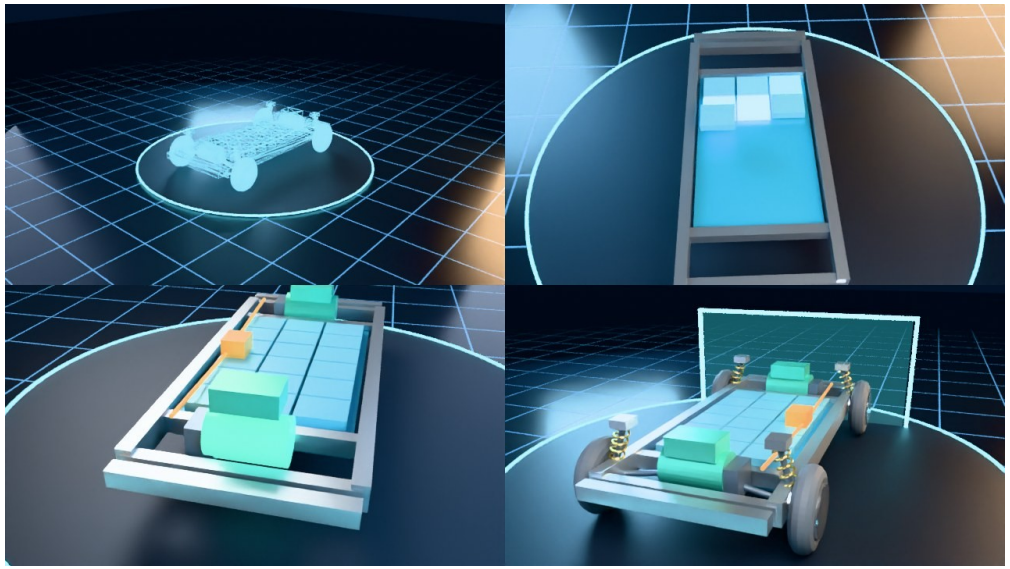
원인 진단과 설정 변경은 Claude Code가 한다. 수정 후 해당 프레임만 다시 확인한다

[입력] 프롬프트 · Claude Code

```
preview 결과에서 세 가지를
고쳐줘.

1) 색이 바랬다
2) 배터리 단계에서 차 뒤쪽이
   잘린다
3) 스캐너가 벽처럼 보인다

고친 뒤 10, 200, 385, 600
프레임만 다시 렌더해줘.
```



Claude Code가 바꾼 설정

- 톤 매핑을 PBR Neutral로 교체
- 카메라를 뒤로, 스캐너는 더 투명하게

STEP 5

본 렌더링은 백그라운드로 돌리고 중간에 멈춰도 이어지게 한다

이미 만든 프레임은 건너뛰는 설정(use_overwrite = False)이 핵심이다

[입력] 프롬프트 · Claude Code

본 렌더링을 백그라운드로 돌려줘.
이미 있는 프레임은 건너뛰게 해서 멈춰도
이어서 렌더되게 하고, 프레임당 15초를
넘으면 샘플을 줄일지 나에게 물어봐줘.

[입력] 명령어 · Claude Code가 터미널에서 실행

```
python scene.py final 91 400
```

[출력] 결과 · renders/

0001.png ~ 0720.png

1~90 프레임
  8샘플, 프레임당 약 20초
91~720 프레임
  5샘플, 프레임당 약 13초

두 구간으로 나눠 실행
전체 약 3시간

04

PART 4 · 후반 작업

자막, 사운드, 최종 인코딩

렌더된 프레임에 공정 흐름 바와 자막을 얹고, 효과음을 합성해 MP4로 묶는다

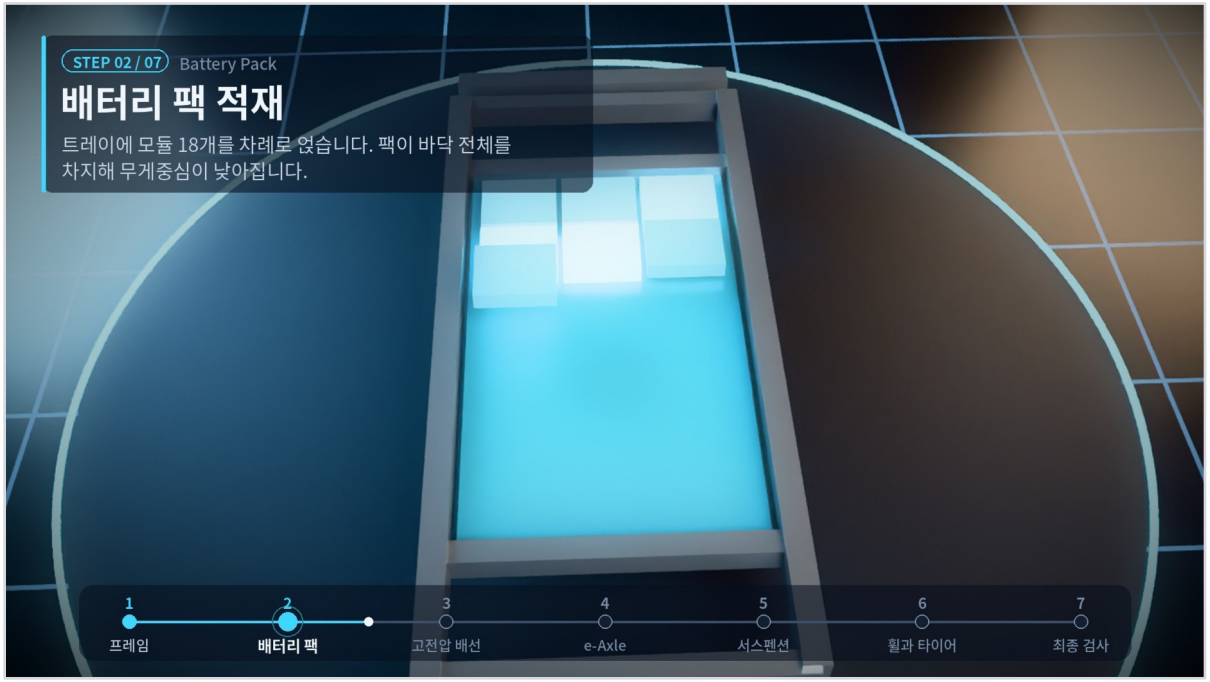
STEP 6

자막과 흐름 바는 렌더 위에 따로 합성해야 고치기 쉽다

문구를 바꿀 때 3시간짜리 렌더를 다시 돌리지 않고 합성만 6분 다시 돌린다

[입력] 프롬프트 · Claude Code

```
renders/ 프레임 위에 자막을
합성하는 comp.py를 만들어줘.
- 1920×1080으로 키우고
  밝은 부분에 은은한 글로우
- 왼쪽 위: STEP 번호, 제목,
  설명 한두 문장
- 아래: 7단계 공정 흐름 바
- 검사 통과 때 PASS 배지
- 끝에 완성 문구
- 폰트: Noto Sans CJK KR
```



효과음은 녹음 없이 파이썬으로 합성해 장면 타이밍에 맞춘다

scene.py가 남긴 events.json의 프레임 번호를 초로 바꿔 소리를 배치한다

[입력] 프롬프트 · Claude Code

```
events.json 타이밍에 맞춰
효과음과 배경음을 합성해
audio.wav를 만들어줘.

날아옴은 휙, 착지는 철컥,
모듈은 틱, 케이블은 지직,
스캔은 상승음, 통과는 차임.
배경은 잔잔한 패드와 박동.
```

소리	합성 방법
휙	대역 통과 필터를 건 잡음
철컥	낮은 사인 감쇠와 금속 배음
틱	2.6kHz의 짧은 사인
지직	톱니파와 잡음
상승음	주파수가 올라가는 사인
차임	C장조 화음의 종소리

FFmpeg로 프레임과 소리를 합쳐 MP4를 만든다

음량을 -16 LUFS로 맞추면 유튜브, SNS, 강의 플랫폼에서 소리 크기가 고르다

[입력] 프롬프트 · Claude Code

```
comp/ 프레임과 audio.wav를 합쳐 MP4를 만들어줘. 음량은 -16 LUFS로 맞추고,
결과 길이와 음량을 측정해서 알려줘.
```

[입력] 명령어 · Claude Code가 실행한 FFmpeg

```
ffmpeg -framerate 24 -i comp/%04d.png -i audio.wav -af loudnorm=I=-16 -c:v libx264 -crf 18 -pix_fmt yuv420p
out.mp4
```

[출력] 결과 · EV_Skateboard_Assembly_Flow.mp4

```
길이 30.0초, 1920×1080, 24fps, 용량 26.5MB
통합 음량 -16.1 LUFS
```

05

PART 5 · 전체 공정

차체를 더한 전기차 조립 공정 영상

차체 모델을 더해 차체 용접부터 도장, 마리지, 의장, EOL 검사까지 아홉 단계를 53초 영상으로 만든다

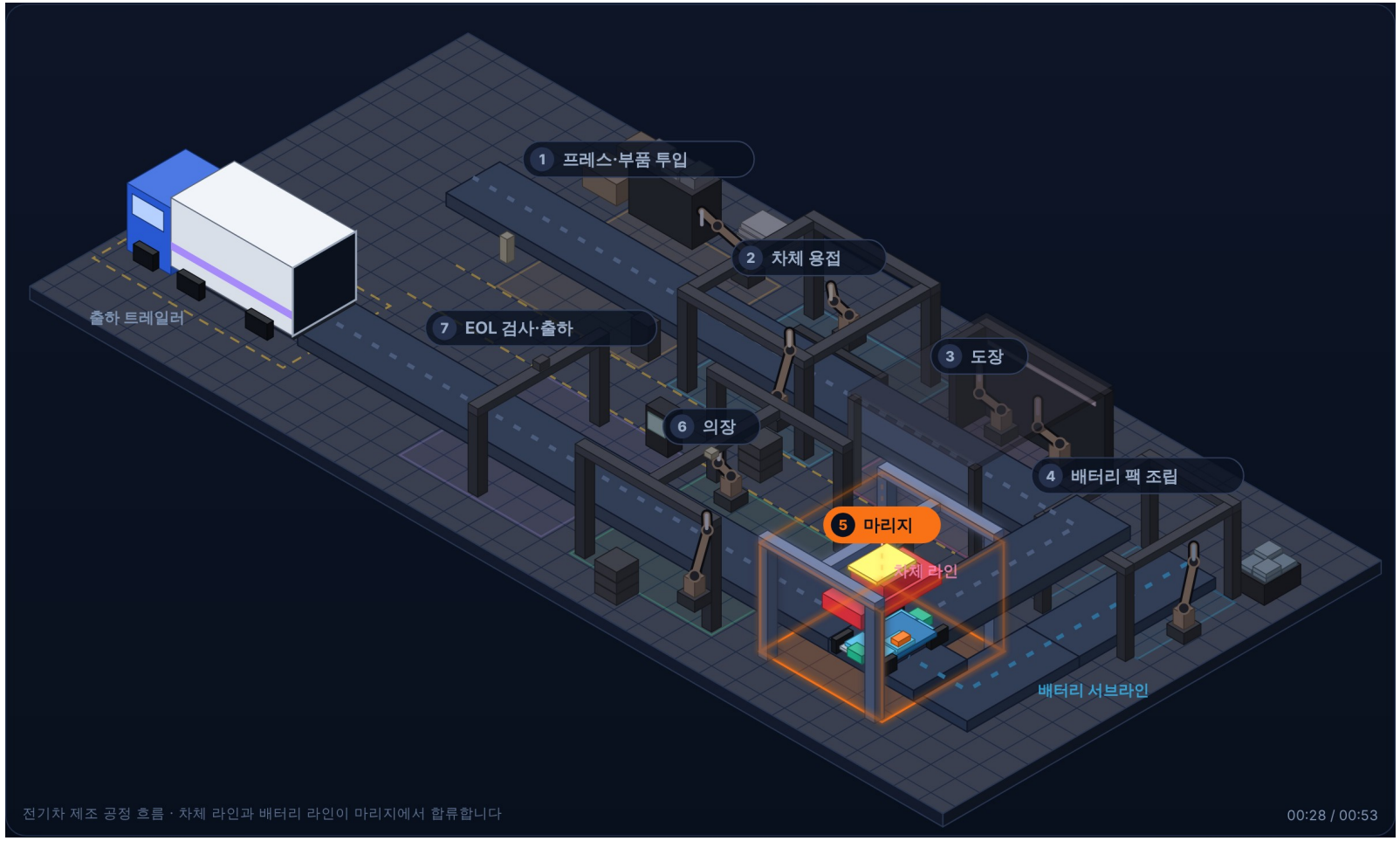
전기차 조립은 두 라인이 마리지에서 만나는 구조다

처음 나오는 공정 용어의 뜻과 영상에서 보여 주는 방법

용어	뜻	영상에서는
차체 용접(BIW, Body in White)	프레스 패널을 용접한 도장 전 강판 차체	오버헤드 컨베이어에 걸린 채 패널이 날아와 붙는다
전착 도장(E-coat)	차체를 도료 탱크에 담가 전기로 입히는 방청 하도	은색 강판이 짙은 회색으로 바뀐다
스케이트보드 플랫폼	배터리 팩과 구동 유닛, 서스펜션, 바퀴를 한 판에 모은 하부 구조	1~5단계에서 408개 부품으로 조립한다
마리지(Marriage)	차체와 하부 플랫폼을 결합하는 공정	컨베이어가 차체를 내려 플랫폼 위에 얹는다
의장(Trim & Final)	유리와 램프, 범퍼 커버, 내장 부품을 다는 공정	유리와 램프, 클래딩이 차례로 날아와 붙는다
EOL(End-of-Line) 검사	라인 끝에서 전장품과 구동계를 확인하는 최종 검사	헤드램프가 켜지고 바퀴가 돈다

공정 흐름도를 먼저 그려 두면 영상의 장면 순서가 정해진다

차체 라인과 배터리 서브라인이 마리지에서 합류하는 7단계 공정 지도(EV_Factory_Flow.html)



공정 영상에는 부품이 나뉜 모델이 필요하다

겉모습이 멋진 완성차 모델은 조립 장면을 만들 수 없다

부품이 무엇으로 나뉘어 있는지가 모델 선택의 첫 기준이다. 차종보다 부품 구조를 먼저 본다

쓸 수 없었던 모델: Sketchfab 포르쉐 911

- 엔진차라서 배터리 팩과 전기 모터 공정을 보여 줄 수 없다
- 차체가 한 덩어리라 부품이 날아와 붙는 장면을 만들 수 없다
- 무료 모델 대부분은 CC-BY라 영상 끝에 제작자 표기가 필요하다

쓴 모델: EV_Crossover_Education.glb

- 메시 1,015개 가운데 플랫폼이 408개, 차체가 607개다
- Body_로 시작하는 이름이 패널, 유리, 램프로 나뉘어 있다
- 플랫폼 부품 이름이 스케이트보드 모델과 같아 함수를 그대로 쓴다

차체 부품은 이름으로 BIW와 의장 두 묶음으로 나뉜다

같은 분석 스크립트로 새 모델의 차체 부품을 공정 단계에 배정한다

[입력] 프롬프트 · Claude Code

```
model.glb(EV 크로스오버)를 불러와
Body_로 시작하는 차체 부품을 정리해줘.

- 패널, 필러, 루프, 후드처럼 강판인 부품은
  차체 용접(BIW) 단계로
- 유리, 램프, 범퍼 커버, 아치, 미러,
  핸들처럼 나중에 다는 부품은 의장 단계로

조각이 많은 패널은 한 장으로 묶어 움직이고,
묶음별 개수를 알려줘.
```

[출력] 결과 · 부품 분류

```
전체 메시 1,015개
(플랫폼 408개 + 차체 607개)

BIW 426개: 사이드 패널 조각 320,
  후드 36, 루프 36, 리어 데크 24, 필러 8
의장 181개: 휠 아치 조각 156,
  유리 4, 램프 2, 범퍼 커버 2, 미러 2
  도어 핸들 4, 이음선 6, 하단 트림 4
도장 색: #176fa4 재질 하나를 공유
앞쪽 = -Y, 차체 하단 z = 0.38
```

아홉 단계를 1,284프레임에 배정한다

24fps에서 1,284프레임은 53.5초다. 플랫폼 다섯 단계는 4초씩으로 줄였다

단계	부품	등장 방식	프레임
1~5 플랫폼	골격, 배터리 팩, 구동 유닛, 서스펜션, 휠	스케이트보드 영상과 같은 방식을 단계마다 4~5초로 압축	73~576
6 차체 용접(BIW)	사이드 패널, 필러, 루프, 후드, 리어 데크	플랫폼 위 2m 높이의 행거에서 패널이 옆과 위에서 날아와 붙는다	577~744
7 도장	차체 재질 하나	강판 은색에서 전착 도장 회색, 상도 청록, 클리어 코트 광택 순으로 바뀐다	745~876
8 마리지	차체 전체(BodyRig)	행거가 차체를 2m 내려 플랫폼에 얹고 빠진다	877~996
9 의장	유리, 램프, 범퍼 커버, 아치, 미러, 핸들	바깥에서 차체 쪽으로 날아와 붙고 착지 순간 빛난다	997~1152
마무리 EOL	헤드램프, 바퀴	램프가 켜지고 바퀴가 2.5바퀴 돈다	1153~1284

다섯 가지 연출이 차체 공정의 흐름을 보이게 만든다

모두 ev2_scene.py 안의 짧은 코드 블록이며, 메타프롬프트 한 줄씩으로 요청했다

오버헤드 행거

BIW에서 마리지까지

차체 부품을 빈 객체 BodyRig의 자식으로 묶고, 플랫폼 위 2m에 걸어 둔다

도장 색 전이

도장

공유 재질의 Base Color에 키프레임 네 개를 찍어 강판에서 상도까지 바꾼다

차체 하강

마리지

BodyRig의 높이 키프레임 두 개로 차체 전체를 한 번에 내린다

착지 발광

의장

단계마다 재질을 복제해 부품이 붙는 순간 발광 강도를 올렸다 내린다

램프와 바퀴

EOL 검사

램프 재질 발광을 깜빡이듯 올리고 휠, 타이어, 허브를 X축으로 돌린다

도장과 마리지는 값 하나를 시간에 따라 보간하는 문제다

색은 두 도장 색 사이를 직선 보간하고, 차체 높이는 부드러운 계단 함수로 내린다

$$c(t) = \big(1 - w(t)\big)\, c_{\text{이전 도장}} + w(t)\, c_{\text{다음 도장}}, \qquad w(t) = \frac{t - t_{\text{시작}}}{t_{\text{끝}} - t_{\text{시작}}}$$

$$z_{\text{차체}}(t) = H_{\text{행거}} \times \big(1 - s(u)\big), \qquad s(u) = 3u^2 - 2u^3, \quad u = \frac{t - t_{\text{하강 시작}}}{t_{\text{착지}} - t_{\text{하강 시작}}}$$

도장: 색 c(t)의 직선 보간

- 강판, 전착 도장, 상도 사이에 키프레임을 두면 Blender가 w(t)로 색을 섞는다

마리지: 높이 z(t)의 S자 하강

- s(u)는 처음과 끝이 느려 차체가 내려앉는 느낌을 준다. 행거 높이 H는 2m

전체 공정 스크립트도 연출을 문장으로 적어 받는다

스케이트보드 스크립트를 출발점으로 차체 단계만 덧붙여 달라고 요청한다

[입력] 프롬프트 · Claude Code (메타프롬프트)

```
스케이트보드 스크립트를 바탕으로 새 모델의
전체 조립 영상 ev2_scene.py를 작성해줘.
- 1920×1080, 24fps, Eevee, 블룸은 컴포지터로
- 플랫폼 5단계는 단계마다 4~5초로 줄여줘
- 차체는 2m 높이 행거에 걸린 채 BIW 용접
- 도장은 강판, 전착 도장, 상도, 클리어 코트
  순서로 색과 광택이 바뀌게
- 마리지는 차체를 내려 플랫폼에 얹고 행거가 빠진다
- 의장 뒤 EOL: 램프 점등, 바퀴 회전
- 자막과 효과음용 timeline.json을 남겨줘
- 이미 렌더된 프레임은 건너뛰게
- PREVIEW=프레임번호면 그 한 장만 렌더
```

[출력] Claude가 생성 · ev2_scene.py

```
STEPS = [...]
  단계, 부품 정규식, 날아오는 방향
BodyRig
  차체 부품의 부모(행거)
glow_copy()
  단계별 재질 복제와 발광
paint 단계
  Base Color 키프레임 4개
marriage 단계
  BodyRig 높이 2m → 0
timeline.json
  단계 제목과 착지 프레임
```

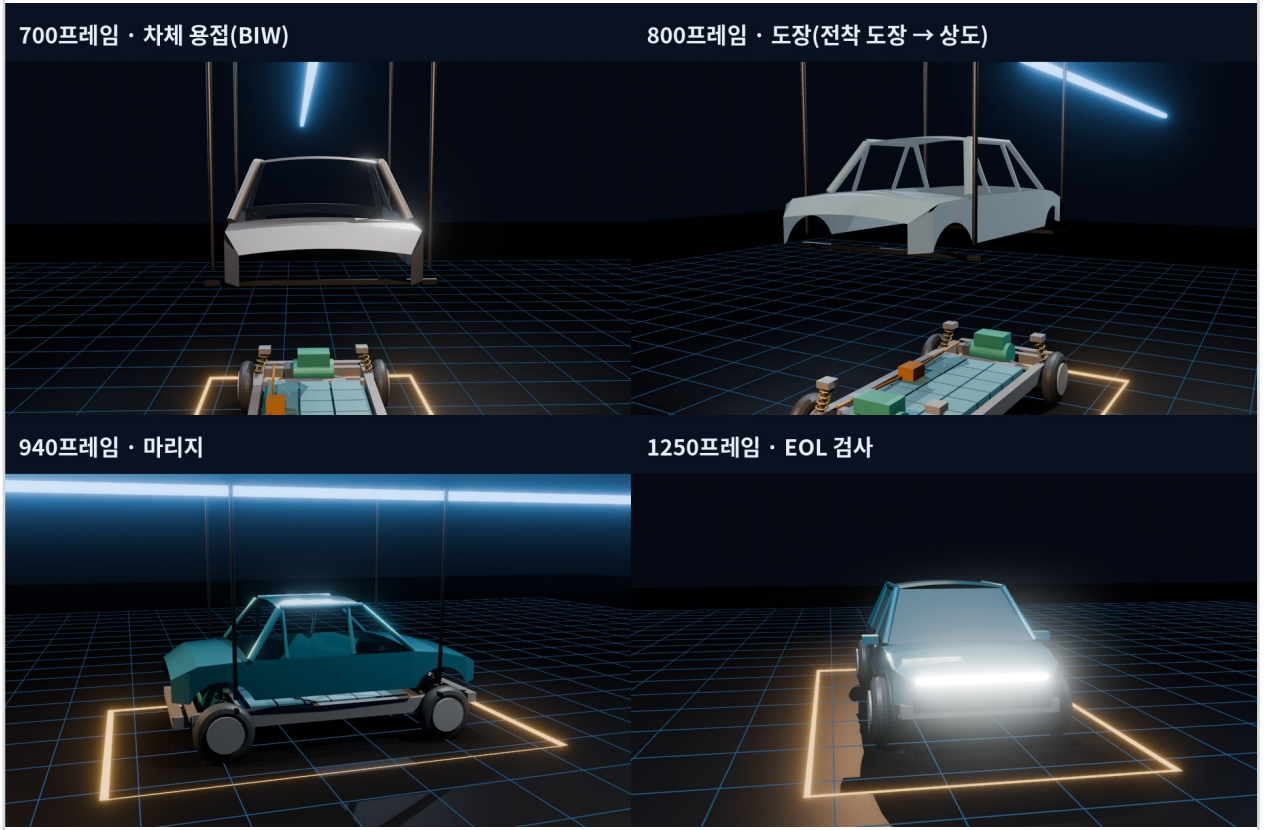
STEP 10

미리보기 다섯 장에서 세 가지를 고쳤다

1080p 4샘플로 단계마다 한 장씩 렌더하면 한 장에 30초, 다섯 장에 3분이면 전체를 본다

고친 문제

- 사이드 유리가 BIW에 나와 이름 규칙을 고쳤다
- 천장 레일이 밝아 발광을 0.5로 낮췄다
- 헤드램프가 화면을 태워 발광을 8로 낮췄다



Eevee도 GPU 없이 돌고, 설정 네 개로 두 배 빨라졌다

1920×1080 한 프레임 실측값이다. bpy 4.2의 Eevee Next는 소프트웨어 GL로 렌더된다

29초

기본 설정
8샘플, 그림자와 AO 켬

21초

그림자 광선 1개
AO(GTAO)도 끔

12.5초

4샘플
글레어 품질 MEDIUM

13~17초

차체 포함 전체 공정
부품 1,015개, 구간마다 다름

알아 둘 점

- EGL 오류 문구가 떠도 렌더는 저장된다. Blender 4.2부터 Eevee에 블룸 체크박스가 없어 컴포지터의 Glare(Bloom) 노드로 대신한다. 이전 강의의 Eevee 실패는 다른 버전과 설정에서의 결과다.

렌더 구간 길이는 작업 시간 제한과 프레임당 시간으로 정한다

백그라운드 작업은 2시간에서 끊긴다. 25% 여유를 두고 구간 하나가 그 안에 끝나게 자른다

$$N_{\text{구간}} = \left\lfloor \frac{0.75 \times T_{\text{제한}}}{t_{\text{프레임}}} \right\rfloor = \left\lfloor \frac{0.75 \times 7200}{17} \right\rfloor = 317$$

$$T_{\text{전체}} = N_{\text{프레임}} \times t_{\text{프레임}} = 1284 \times 15 \approx 19{,}300\,\text{초} \approx 5.4\,\text{시간}$$

구간 길이 N구간

- 프레임당 17초면 317프레임이다. 전체를 네 등분한 321프레임으로 정했다

전체 시간 T전체

- 네 구간을 차례로 걸면 약 5시간 반, 자막과 소리 합성에 15분이 더 든다

긴 렌더는 구간으로 나눠 차례로 건다

구간이 끝날 때마다 다음 구간을 건다. 끊겨도 이미 만든 프레임은 남는다

[입력] 프롬프트 · Claude Code

```
1,284프레임을 네 구간으로 나눠
렌더해줘. 구간 하나는 2시간 안에
끝나야 한다. 끝나면 빈 파일을 지우고
다음 구간을 걸고, 마지막 구간 뒤에는
자막, 소리, MP4까지 만들어줘.
```

[입력] 명령어 · Claude Code가 터미널에서 실행

```
./render_chunk.sh 1 321
```

[출력] 결과 · renders/

```
frame_0001.png ~ frame_1284.png

1구간 1~321
  프레임당 17초, 약 1시간 35분
2~4구간 322~1284
  프레임당 13~15초, 구간당 약 1시간 20분

이후 overlay.py, sfx_mix.py, FFmpeg
EV_Full_Assembly.mp4, 53.5초
```

RESULT

완성 영상은 아홉 단계 전체 공정을 53.5초에 담았다

왼쪽 위에서 오른쪽 아래 순서로 구동 유닛, BIW, 도장, 마리지 두 장, 의장 두 장, EOL 검사



전체 공정으로 늘리며 만난 문제 다섯 가지

이번 확장 제작 로그에서 뽑은 사례다. 원인을 찾고 고친 것은 모두 Claude Code다

증상	원인	해결
렌더가 2시간에서 끊김	백그라운드 작업 시간 제한	321프레임씩 네 구간으로 나눠 실행
make_final.sh 실행 거부	스크립트에 실행 권한이 없음(Permission denied)	chmod +x로 권한을 주고 다시 실행
사이드 유리가 BIW에 등장	정규식이 Body_Side_Glass까지 잡음	Side_ 뒤에 숫자가 오는 이름만 잡도록 수정
Eevee에 블룸 설정이 없음	Blender 4.2 Eevee Next에서 제거됨	컴포지터 Glare(Bloom) 노드로 대체
뒷문에 7 모양 검은 선	모델에 들어 있는 도어 이음선 부품	그대로 두거나 그 부품만 숨겨 다시 렌더

제작 중 만난 문제 다섯 가지는 모두 Claude Code가 진단했다

실제 제작 로그에서 뽑은 사례다. 수강생도 같은 문제를 만날 수 있다

증상	원인	해결
스크립트가 시작하자마자 멈춤	파일명 inspect.py가 파이썬 표준 모듈과 충돌	probe_glb.py로 이름 변경
Eevee 렌더 실패	GPU 없는 서버라 EGL 오류	Cycles CPU로 전환
프레임당 19초	홀로그램 투명도와 바닥 반사	91프레임부터 5샘플, 반사 1회
렌더를 멈추자 셸도 종료	pkill -f가 같은 문자열의 셸까지 종료	프로세스 번호로 지정해 종료
범퍼와 프레임 사이 간격	원본 모델의 배치	영상은 그대로 두고 모델 수정은 별도

사람은 기획하고 판정하며, 실행은 Claude Code가 맡는다

이번 제작에서 실제로 나뉜 일

사람의 몫

- 보여줄 공정과 순서를 정한다
- 미리보기에서 어색한 장면을 짚는다
- 렌더 시간과 화질의 균형을 승인한다
- 완성 영상을 보고 최종 판정한다

Claude Code의 몫

- Blender와 FFmpeg 설치와 점검
- 모델 분석과 scene.py 작성
- 렌더 실행, 속도 측정, 설정 조정
- 자막과 소리 합성, MP4 인코딩
- 오류 원인 진단과 수정

DISCUSSION

같은 방법을 우리 업무 영상에 옮길 때 판단할 것들

정답보다 선택의 이유를 이야기한다

Q1	우리 제품 공정을 영상으로 만들려면 어떤 3D 모델이 필요하고, 부품 이름은 어떻게 나뉘어 있어야 하는가?	**모델 준비**
Q2	렌더 시간이 5시간일 때 샘플, 해상도, 길이, 효과 중 무엇부터 줄이겠는가? 구간을 어떻게 나누겠는가?	**시간 예산**
Q3	미리보기 단계를 건너뛰고 바로 본 렌더링을 걸면 어떤 비용이 생기는가?	**검수**
Q4	이 파이프라인을 영상 시리즈로 확장한다면 무엇을 템플릿으로 남기겠는가?	**재사용**

SUMMARY

모델 두 개와 프롬프트 열두 개로 공정 영상 두 편을 만들었다

이번 강의의 정리

1

부품 이름이 공정 순서를 알려준다

GLB 부품을 이름 패턴으로 묶고, 묶음마다 등장 방식과 프레임을 배정한다.

2

움직임은 fly_in 함수 하나로 통일한다

출발 오프셋과 최종 위치 두 키프레임, EXPO와 BACK 보간이면 충분하다.

3

미리보기, 본 렌더링, 합성을 분리한다

고칠 일이 생기면 가장 싼 단계만 다시 돌리고, 긴 렌더는 시간 제한에 맞춰 구간으로 나눈다.

다음 단계: GPU가 있는 PC에서 고샘플로 다시 렌더하고, 용접 로봇 장면을 더해 본다